Up to this point, you have been using model binding to populate simple properties from incoming form values. As the number of form fields grows, the `PageModel` class will start to creak with either a long list of properties, all decorated with the `BindProperty` attribute, or a large number of parameters applied to a handler method. Fortunately, model binding also works with complex objects, so the properties to be bound can be encapsulated in an object that can be exposed as a property of the `PageModel` or a parameter for the handler method. Some developers refer to the object as an *input model* when it is used in this manner, and it provides an excellent way to limit the scope of the binding target. To see how this works, you will start expanding the functionality of the CityBreaks application.

Each of your cities belongs to a country. You will produce a form that enables you to capture country data using an input model. It will be a very simple input model, but it will do for the sake of example.

Create a folder named Models in the root of the CityBreaks application, and add a C# class named Country.cs to it. Amend the code, so it is the same as the following listing.

Listing 5.8 The `Country` class

```
public class Country
{
    public string CountryName { get; set; }
    public string CountryCode { get; set; }
}
```

The `CountryName` property represents the name of the country, and the `CountryCode` property represents the two-character ISO 3166-1 alpha-2 code for each country. For example, the United States is `us`, and the United Kingdom is `gb`. You can find a full list of these codes from Wikipedia: